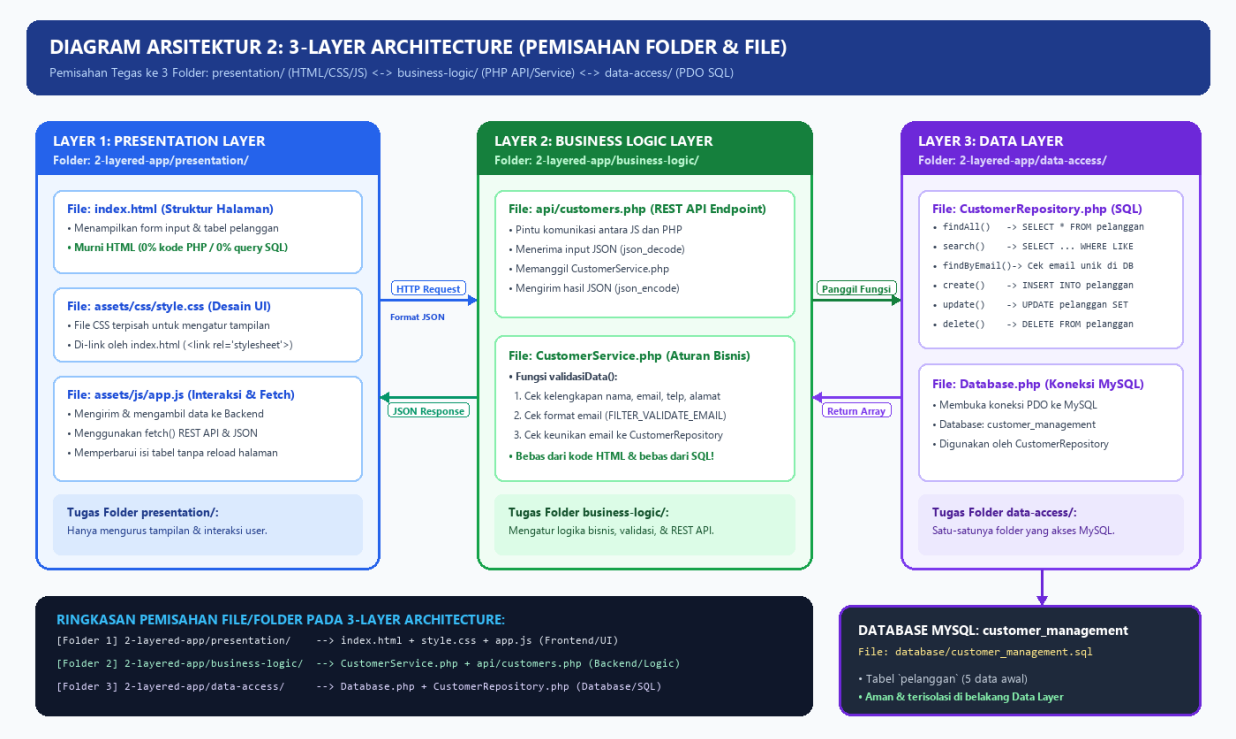
LAPORAN TUGAS KELOMPOK

ANALISIS DAN IMPLEMENTASI PERBANDINGAN LAYERED VS NON-LAYERED WEB APPLICATION ARCHITECTURE

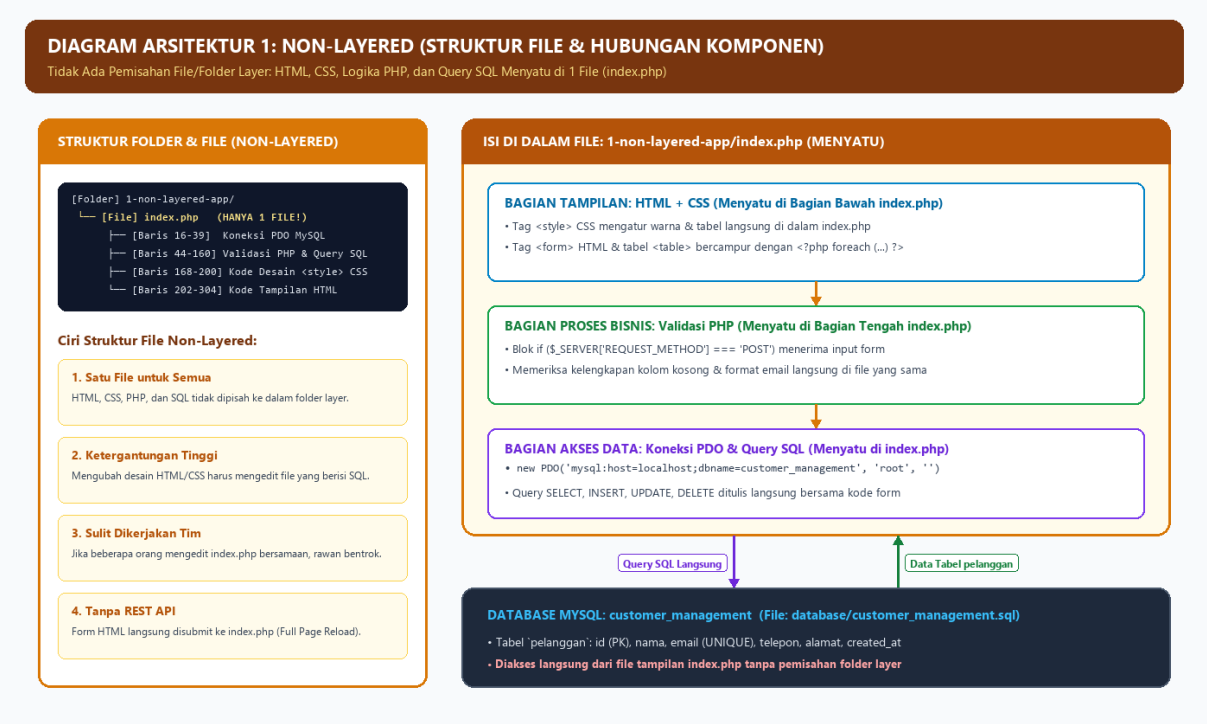
Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)



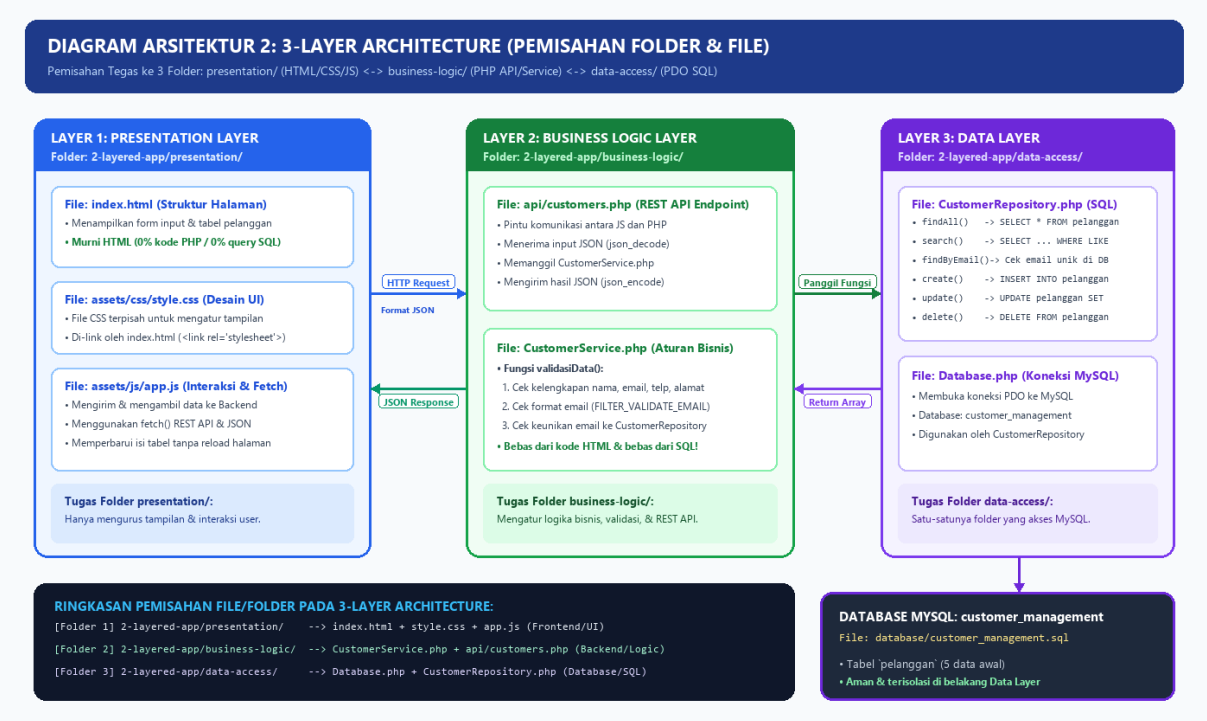
Mata Kuliah	Pemrograman Web / Arsitektur Aplikasi Web
Topik Kajian	Perbandingan Non-Layered Architecture (1 File Monolitik) vs 3-Layer Architecture
Studi Kasus	Customer Management System (Tambah, Tampil, Edit, Hapus, Pencarian, & Validasi Email Unik)
Teknologi	PHP Native (Non-Framework), MySQL (PDO), HTML5, CSS3, JavaScript Fetch API (REST & JSON)

BAB 1. DIAGRAM ARSITEKTUR (PEMISAHAN FILE/FOLDER & HUBUNGAN KOMPONEN)

Perbedaan paling mendasar antara **Non-Layered Architecture** dan **3-Layer Architecture** terletak pada bagaimana file/folder dipisahkan serta bagaimana kode **HTML, CSS, JavaScript, PHP, dan SQL** saling berhubungan.



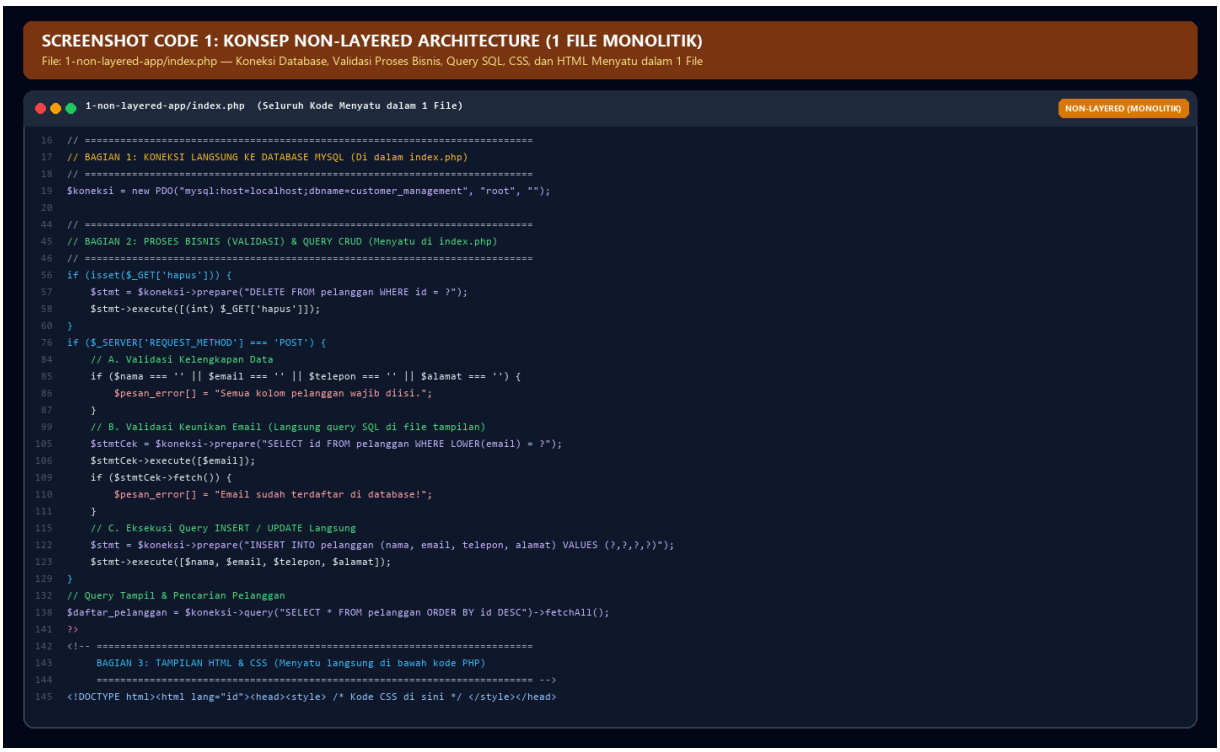
Gambar 1.1 Diagram Arsitektur Non-Layered: Seluruh Komponen (HTML, CSS, PHP, SQL) Menyatu dalam 1 File index.php



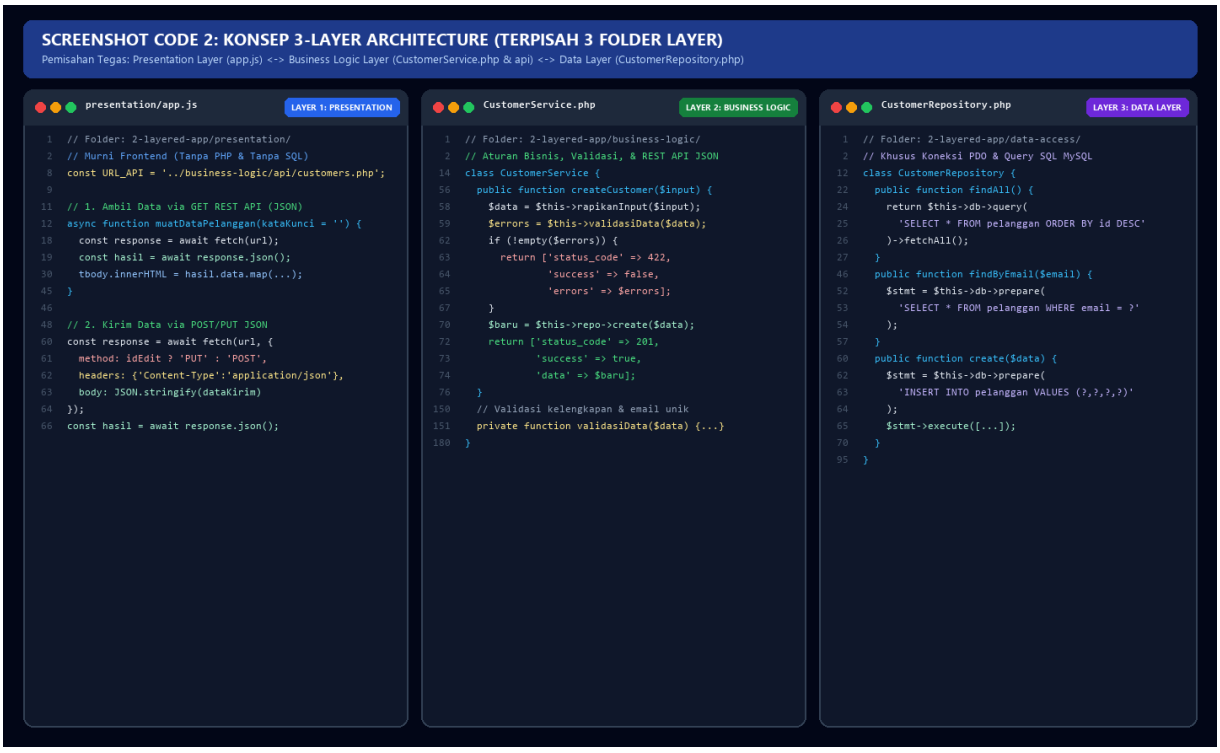
Gambar 1.2 Diagram Arsitektur 3-Layer: Pemisahan Tegas ke dalam 3 Folder Layer (presentation, business-logic, data-access)

BAB 2. SCREENSHOT CODE KEDUA KONSEP ARSITEKTUR

Berikut adalah perbandingan langsung struktur kode program pada kedua pendekatan untuk fitur yang sama (Customer Management System):



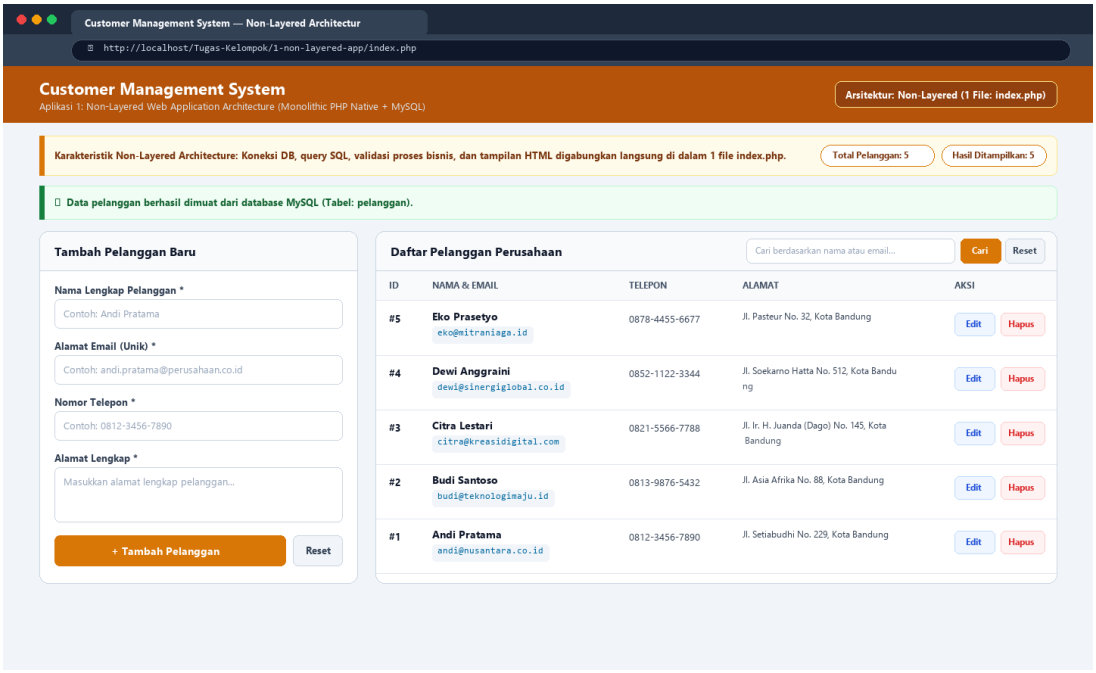
Gambar 2.1 Screenshot Code Non-Layered Architecture: Koneksi DB, Validasi, Query SQL, CSS, dan HTML Menyatu di 1 File (index.php)



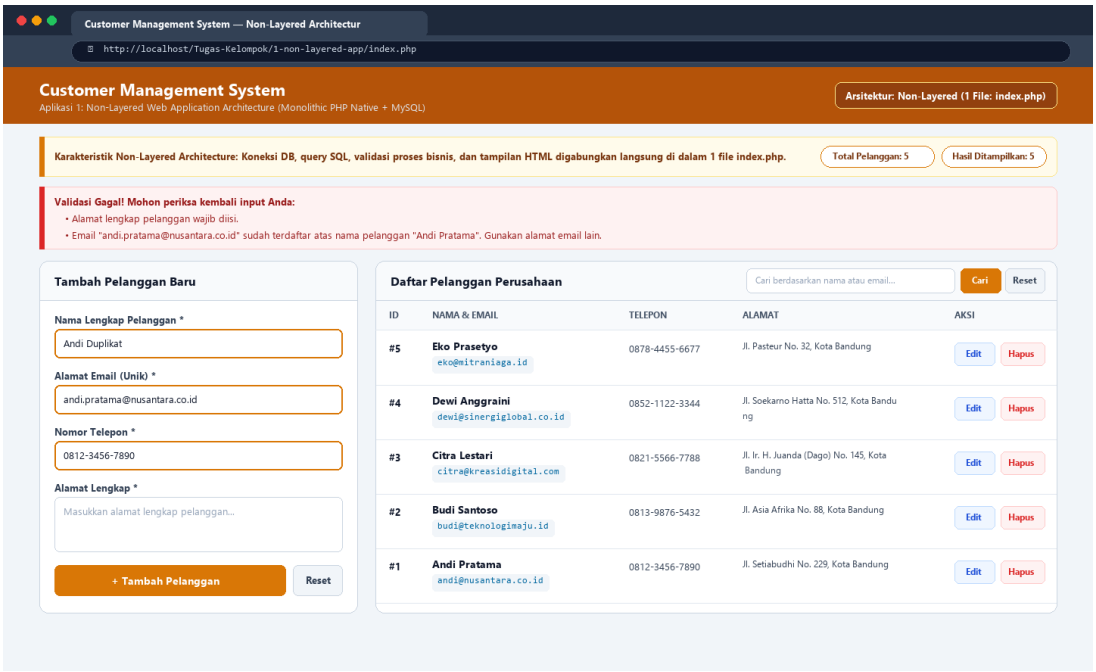
Gambar 2.2 Screenshot Code 3-Layer Architecture: Kode Terpisah ke dalam Presentation Layer (JS), Business Logic Layer (Service), dan Data Layer (Repository)

BAB 3. SCREENSHOT TAMPILAN KEDUA APLIKASI WEB

3.1 Tampilan Aplikasi 1 (Non-Layered Architecture)

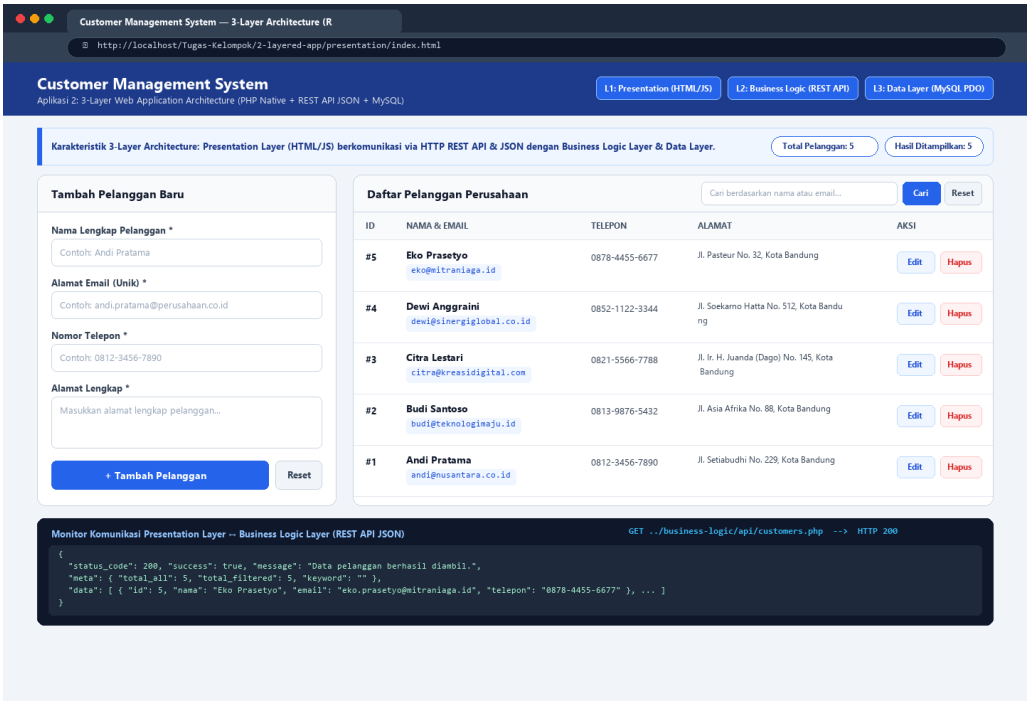


Gambar 3.1 Tampilan Utama & Daftar Pelanggan Aplikasi 1 (Non-Layered Architecture)

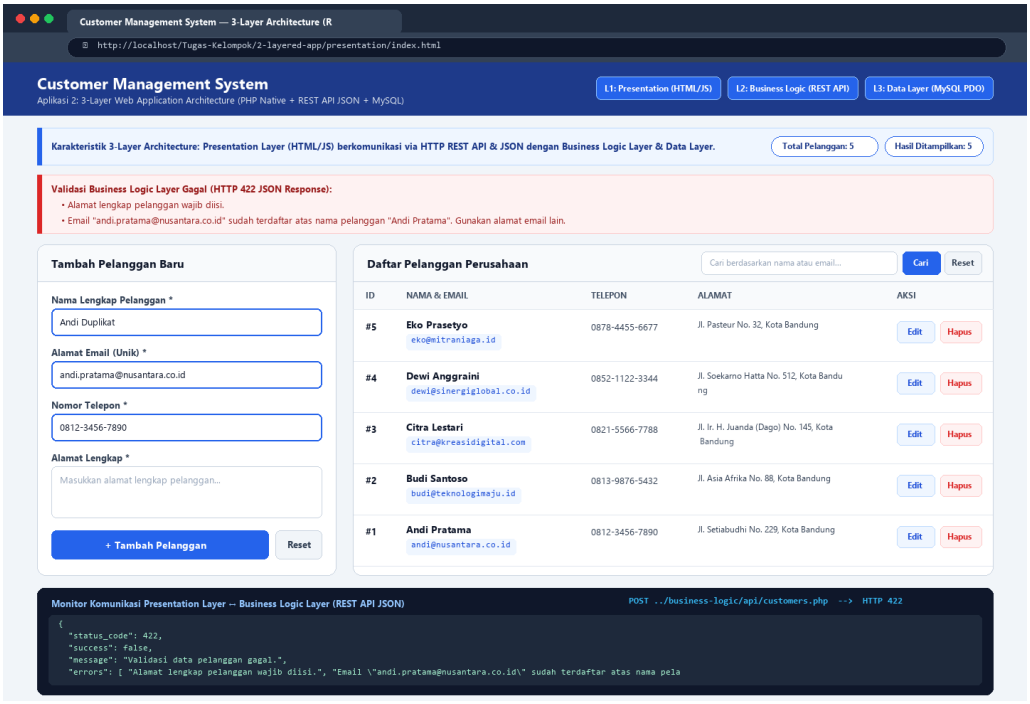


Gambar 3.2 Tampilan Validasi Kelengkapan & Keunikan Email Aplikasi 1 (Non-Layered Architecture)

3.2 Tampilan Aplikasi 2 (3-Layer Architecture dengan REST API & JSON)



Gambar 3.3 Tampilan Utama Aplikasi 2 (3-Layer Architecture) beserta Panel Monitor REST API JSON (HTTP 200)



Gambar 3.4 Tampilan Validasi Business Logic Layer (HTTP 422 JSON Response) pada Aplikasi 2

BAB 4. TABEL HASIL PENGUJIAN DAN PERBANDINGAN AKHIR

Berdasarkan hasil uji coba pembangunan dan pengoperasian kedua aplikasi Customer Management System, berikut adalah **Tabel Hasil Pengujian dan Perbandingan** yang merangkum bagaimana hasil uji coba pada pendekatan tanpa layer vs menggunakan 3-layer, pendekatan mana yang lebih unggul pada setiap unsur, beserta alasannya:

No	Unsur yang Diuji Coba	Tanpa Layer (Non-Layered)	Menggunakan 3-Layer	Lebih Unggul	Alasan Mengapa Lebih Unggul
1	**Struktur Pemisahan File & Folder (Hubungan HTML, CSS, & PHP)**	Tidak dipisahkan. HTML, CSS, validasi PHP, dan query SQL digabung dalam 1 file (index.php).	Dipisahkan tegas ke 3 folder: presentation/ (HTML/CSS/JS), business-logic/ (PHP API), & data-access/ (SQL).	**3-Layer Architecture**	Kode jauh lebih rapi dan terorganisasi karena setiap file hanya memiliki satu tanggung jawab spesifik.
2	**Kecepatan Pembuatan Awal (Initial Development Speed)**	Sangat cepat dibuat di awal karena programmer cukup menulis langsung di 1 file tanpa membuat REST API.	Membutuhkan waktu awal lebih lama karena harus membuat kontrak JSON, Service, dan Repository di 3 folder.	**Non-Layered Architecture**	Untuk aplikasi yang sangat kecil/sederhana, menulis di 1 file lebih praktis tanpa overhead pembuatan banyak file.
3	**Pengerjaan secara Individu vs Tim (Teamwork & Kolaborasi)**	Cocok untuk 1 orang (individu), tetapi buruk untuk tim karena semua orang mengedit 1 file index.php yang sama.	Sangat ideal untuk kerja tim: anggota A mengerjakan folder presentation/, B di business-logic/, C di data-access/.	**3-Layer Architecture**	Pembagian tugas kelompok menjadi paralel dan tidak saling menimpa kode (bebas merge conflict di Git).
4	**Kemudahan Penelusuran Kesalahan (Debugging / Error Tracing)**	Sulit ditelusuri saat kode membesar karena harus mencari apakah error berasal dari HTML, validasi, atau SQL di 1 file.	Sangat mudah dilacak: error tampilan cek di presentation/, error validasi di CustomerService, error SQL di Repository.	**3-Layer Architecture**	Sumber bug langsung terisolasi per folder layer sehingga proses perbaikan kesalahan jauh lebih cepat.
5	**Kecepatan & Kenyamanan Interaksi (Pengalaman Pengguna / UX)**	Setiap kali tambah/cari/hapus data, browser harus memuat ulang (reload) seluruh halaman HTML dari awal.	Lebih cepat & responsif karena hanya mengirim/menerima data JSON via fetch() tanpa me-reload seluruh halaman.	**3-Layer Architecture**	Pertukaran data menggunakan REST API & JSON lebih ringan dibandingkan me-render ulang seluruh halaman HTML.
6	**Kemudahan Pemeliharaan Desain (Maintainability UI vs Database)**	Berisiko tinggi: saat mengubah desain tabel HTML/CSS, programmer bisa tidak sengaja merusak kode query SQL di atasnya.	Sangat aman: perubahan desain di index.html / style.css sama sekali tidak menyentuh file PHP maupun query SQL.	**3-Layer Architecture**	Perubahan pada tampilan antarmuka tidak memengaruhi logika bisnis maupun struktur akses database.
7	**Kemudahan Pengembangan Fitur (Scalability / Fitur Bertambah)**	Semakin banyak fitur ditambahkan, file index.php semakin panjang dan rumit (menjadi spaghetti code).	Sangat mudah dikembangkan: penambahan fitur baru cukup menambah fungsi secara modular di masing-masing layer.	**3-Layer Architecture**	Struktur modular menjaga kode tetap bersih meskipun fitur aplikasi terus bertambah besar.
8	**Keamanan & Penggunaan Ulang (Security & Reusability API)**	Query database berada di file tampilan dan tidak bisa dipakai oleh aplikasi Mobile (karena outputnya HTML).	Database terisolasi di Data Layer, dan endpoint REST API JSON (customers.php) bisa langsung dipakai Web & Mobile.	**3-Layer Architecture**	Satu Backend REST API JSON dapat melayani berbagai platform sekaligus (Web maupun Android/iOS).

BAB 5. JAWABAN PERTANYAAN DISKUSI DAN KESIMPULAN

1. Mana yang lebih mudah dibuat?

Non-Layered Architecture lebih mudah dan cepat dibuat di awal (khususnya jika dikerjakan sendiri untuk aplikasi kecil), karena seluruh koneksi database, validasi, query SQL, dan tampilan HTML cukup ditulis dalam 1 file index.php.

2. Mana yang lebih mudah diperbaiki ketika terjadi kesalahan (bug)?

3-Layer Architecture jauh lebih mudah diperbaiki. Karena adanya pemisahan folder per layer, penelusuran kesalahan menjadi sangat cepat: error tampilan dicari di folder presentation/, error validasi di CustomerService.php, dan error database di CustomerRepository.php.

3. Mana yang lebih mudah dikembangkan ketika fitur bertambah?

3-Layer Architecture jauh lebih mudah dikembangkan. Penambahan fitur baru maupun kerja sama tim dapat dilakukan per folder layer tanpa mengganggu layer lain, serta REST API JSON-nya dapat langsung dipakai oleh aplikasi lain (seperti Mobile).

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Kode menjadi sangat panjang dan tercampur aduk (*spaghetti code*), perubahan desain HTML/CSS rawan merusak query SQL, sulit dikerjakan bersama oleh tim (rawan bentrok kode), dan sulit ditelusuri saat terjadi bug.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk aplikasi yang sangat sederhana, prototipe cepat, atau tugas individu berskala kecil, pendekatan tanpa layer justru praktis dan cepat. Namun untuk aplikasi yang dikerjakan secara tim dan terus berkembang, **3-Layer Architecture jauh lebih unggul**.

Kesimpulan Akhir

Berdasarkan hasil pengujian perbandingan di atas, **Non-Layered Architecture** unggul dari segi kecepatan pembuatan awal untuk aplikasi kecil yang dikerjakan individu. Sebaliknya, **3-Layer Web Application Architecture** unggul mutlak dalam kerapian pemisahan file/folder, kolaborasi tim, kemudahan penelusuran kesalahan (*debugging*), keamanan database, kemudahan pengembangan fitur baru, serta kecepatan interaksi antarmuka berkat penggunaan **REST API dan JSON**.